Asmus Nyeholt - How old are you: utilizing social media to estimate users' age | Machine Learning Final report

1. Mid Term Milestone Goals(proposal from 1/16/26)

-Have the model ready and begin training
-Data cleaned and trimmed
-Background section describing the problem
-Description of the methods used and experiments detailing the results

2. Dataset

Instagram user activity and personal info data set

See .ipynb file for more details relating to how we reformatted the data set

Google Colab Code Link

3. Background section describing the problem

The majority of time spent on the final project was preparing the data for the models. There were many features that added noise to our dataset that we had to remove. Additionally we removed 90% of the instances due to an abundance of instances. This was done randomly as we wanted the results to be random every time we began working with the models .

We have a very large data set and we need to format the data with categorical features into binary features. We will use one hot encoding for this process. Along with a label encoder for the categorical binary features. In addition to our data set being so large, we use a correlation matrix to calculate the impact of certain features and whether or not we will use them in our model.

Once the data set has been organized appropriately, we then begin stochastic gradient descent regression to find any variance, then use cross validation to see the accuracy of our model with the linear regression. This allowed us to better understand which features we have that can be used for the best results. This helps slow down computational time wasted and allow for better results.

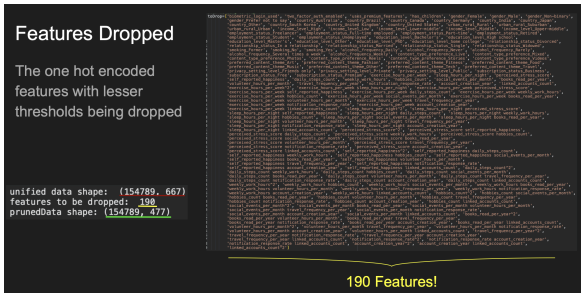
Preprocessing

A random set of 10% of the total set of instances were used as data to evaluate and train the models. This cut the number of instances from ~1.5 million to ~1.5 thousand and greatly increased the speed at which operations could be performed, without significantly altering model results.

Features were separated into three groups by type. Numerical features were transformed by a PolynomialFeatures of degree 2, in an attempt to better fit the label, and subsequently standardized. Categorical features that represented a boolean value were encoded to hold a 0 or 1 using a LabelEncoder, while those with tabular data were transformed using OneHotEncoder.

The correlation of every feature in the dataset to the label was generated using .corr() and, because models tend to underfit the data, all features with a correlation less than a threshold were removed to reduce noise and improve the fit.

Instagram user activity and personal lifestyle dataset. We removed a list of features that is extremely long.



We removed these due to their low correlation (they were below the threshold of .007). We were comfortable removing features due to the abundance of features created during the use of PolynomialFeatures and OneHotEncoder.

Linear Models

We used R^2 for scoring as it is the baseline used by most regression models and is a function of the mse, the value we were attempting to decrease.

Linear Regression: On both the training and test set, the model scored ~63% accuracy. As the most basic model, we used this as a baseline and most of our time was spent attempting to find a model and hyperparameters that could surpass LinearRegression.

SGDRegression: We initially expected more complex models to perform better than LinearRegression, but SGDRegression scored significantly lower, at ~61% accuracy. This model introduced a small amount of variance through its hyperparameters, but we found the default parameters gave the greatest score.

Elastic Net: In an attempt to add more regularization, we pivoted to elastic net regression. The ratio of lasso to ridge regression was the primary hyperparameter that we tested. Regardless of its value, this model greatly underperformed relative to other models we tested. The higher the ratio of lasso regression, the higher the model scored. With its default hyperparameters, this model scored ~55% accuracy. This was our worst performing model.

Lasso Regression: Lasso Regression represents the ceiling of what we could do with ElasticNet. It highlights the importance of regression in the models we use, but it does not perform well, scoring ~57%.

Ensemble Models

Random Forest Regressor: The random forest regressor model had 50 estimators and a maximum depth of 5 with a score of 64% on the test set. This was the first model that surpassed LinearRegression, but the difference between the two was negligible. This model also took an absurd amount of time to train; considering how little the

improvement was compared to the quick-to-train LinearRegression, it was not worth it.

Bagging Regressor: We used this model in an attempt to improve upon the accuracy of LinearRegression by using it as the base for Ensemble models. 10 estimators were used for the BaggingRegressor. Unfortunately it performed almost the same as the LinearRegression.

AdaBoost Regressor: This regressor will improve off of the previous predictors underfitting mistakes gradually improving the model. We chose LinearRegression as our estimator, as it was our best linear model. This was done with 50 estimators and scored ~61% accuracy on the test set, which was only 1% below the training set. This model was the worst of our ensemble models despite the number and type of the estimators used.

ANN Neural Network (Sequential API):The neural network seemed to be a model with more complication compared to the rest due to the amount of time spent covering it in class. Assignment 5 assisted us through the creation of our neutral network. Normalization was used for the input layer, then 4 hidden layers using relu as our activation function alongside 250 neurons. Then the output layer did not have an activation function, as regression does not need one. We introduced early stopping for the possibility of our model being expensive in computational time. Early stopping focused on the mse, which was our loss function as well. If the mse changed more than .002 after 3 epochs it would stop early. The model did not stop early once during our experiments. Once the model reached 10 epochs, it would consistently improve by ≈.2 MSE each epoch. The only downside to this model is the fact each epoch takes ≈17 seconds to complete. This being said, if we wanted to decrease the mse, we would have to train the model with many more epochs. We agreed that this wasn't necessary as our score with 30 epochs was already better than our base case linear regression with a score of 66% R squared value found from using r2_score in sklearn.metrics.